

JIS COLLEGE OF ENGINEERING

Block A, Phase III, Kalyani, Nadia - 741235, West Bengal
Department of Computer Science & Engineering

MAJOR PROJECT - I | REVIEW 2

Mid-Term Project Evaluation — Experimental Result

***Blockchain-Assisted Distributed AES Key Lifecycle Management Framework for Secure
Healthcare Internet of Things***

Submitted By

Niraj Kumar (Team Leader) — 123231203090

Piyush Kumar — 123231203093

Kanhaiya Kumar — 123231203069

Prashant Kumar — 123231203094

Guided By

Dr. Bikramjit Sarkar

Professor, Department of Computer Science & Engineering

Project ID: T4-2026-030

Review Date: 4th Week of September 2026

1. Project Recap

The project develops a Blockchain-Assisted Distributed AES Key Lifecycle Management Framework for Healthcare IoT. Patient medical records are encrypted using AES-256 and stored in a relational database, while a blockchain-enabled smart contract layer (planned for a later phase) will manage cryptographic key metadata and maintain a tamper-resistant audit trail. The complete system architecture was presented in Review 1.

2. Scope of This Review

This Mid-Term Review presents the complete, independently functioning AES key-lifecycle management engine — the security core on which the rest of the system (blockchain integration and frontend) will be built. The team deliberately prioritized building and rigorously testing this core before extending into blockchain and UI work, since those layers depend on the correctness of the key-management engine underneath them.

Accordingly, this review covers Phases 1 through 14 of the development roadmap, corresponding to project foundation, security implementation, complete key lifecycle management, authentication and authorization, audit logging, and transactional database integrity. Blockchain integration and the frontend application are scheduled for subsequent reviews and Major Project II, as outlined in Section 7.

3. Implementation Summary (Phases 1–14)

Phase	Module	Status	Summary
1	Project Foundation	● Completed	Spring Boot + Java project structure, layered architecture (Controller → Service → Repository).
2	Core Setup	● Completed	Maven dependencies, application configuration, MySQL connectivity via JPA/Hibernate.
3	Key Management (Base)	● Completed	AES-256 key generation, key metadata model (keyId, version, status, algorithm).
3.1 / 3.2	Extended Key Functionality	● Completed	AES-GCM mode with random IV; metadata/response DTOs excluding sensitive key material.
5	Security Foundation	● Completed	AES-256-GCM encryption verified — confidentiality + integrity via authentication tag.
6	Key Protection	● Completed	Master-key-based protection layer (KeyProtectionService) — plaintext AES keys never stored directly.
7	Key Lifecycle	● Completed	Full state machine: CREATE → ACTIVE → ROTATE / REVOKE, with versioning.
8	Exception Architecture	● Completed	Centralized GlobalExceptionHandler, custom exceptions, standardized ErrorResponse.
9	Authentication & Authorization	● Completed	JWT-based login, BCrypt hashing, role-based access control (USER / ADMIN).
10	Audit Logging	● Completed	AuditLoggingFilter records who/what/when/status for every request.
11	Key Expiration	● Completed	Scheduler-based automatic expiry check (60s interval) using expiresAt field.
12	Key Archival	● Completed	Inactive key versions archived instead of deleted; active version protected.
13	Emergency Recovery	● Completed	Controlled PENDING → APPROVED → COMPLETED recovery workflow.
14	DB Transactions	● Completed	@Transactional-backed atomic operations verified via key rotation test.

Overall: 14 / 14 planned phases for this review are complete and verified. All code has been committed and pushed to the team's GitHub repository.

4. Technical Implementation & Experimental Verification

4.1 AES-256-GCM Encryption (Phase 5)

Encryption uses AES/GCM/NoPadding with a 256-bit key and a random 12-byte Initialization Vector (IV) generated per operation using SecureRandom. GCM mode provides both confidentiality and integrity: any tampering with the ciphertext is detectable via the authentication tag, in addition to standard confidentiality guarantees.

Plaintext + AES-256 Key + Random 12-byte IV → AES-GCM → Ciphertext + Authentication Tag

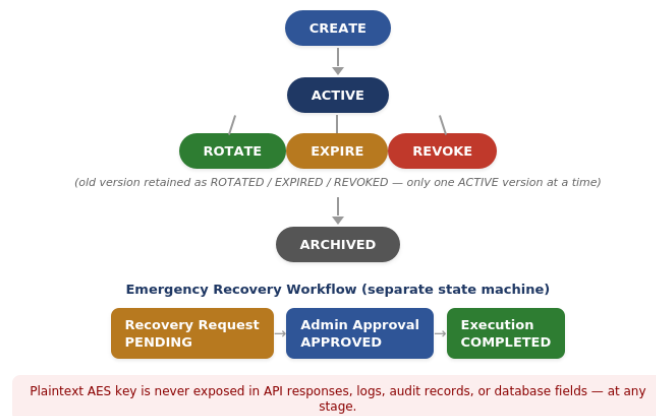
Verification: Encryption and decryption were validated with unit tests confirming that (a) identical plaintext encrypted twice with the same key produces different ciphertext (due to random IV), and (b) decryption correctly recovers the original plaintext.

4.2 Key Protection Layer (Phase 6)

Rather than storing generated AES keys directly, each key is passed through a KeyProtectionService that encrypts it using a separate master key (supplied via environment configuration, not hard-coded) before persistence. Only the protected form of the key is ever written to the database. API responses (via KeyMetadataResponse DTOs) intentionally exclude both the protected and plaintext key material — only non-sensitive metadata such as keyId, algorithm, version, and status is returned.

4.3 Key Lifecycle Management (Phases 7, 11, 12, 13)

The system implements a complete key lifecycle state machine covering normal operation, time-based expiration, historical preservation, and controlled emergency recovery:

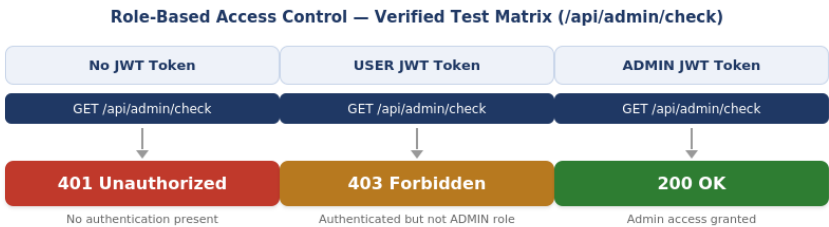


- Rotation: old key version moves to ROTATED, a new version is generated as ACTIVE — verified so exactly one ACTIVE version exists per keyId at any time.
- Expiration: a scheduler checks the database every 60 seconds and automatically transitions keys past their expiresAt timestamp to EXPIRED.
- Archival: inactive versions (ROTATED / EXPIRED / REVOKED) can be archived for historical retention; the system blocks archival of a currently ACTIVE version.
- Emergency Recovery: uses an independent RecoveryStatus state machine (PENDING → APPROVED → COMPLETED) requiring explicit admin approval before execution; invalid transitions (e.g., executing a PENDING request) are rejected.
- Security invariant: the plaintext AES key is never exposed via API responses, logs, audit records, or database fields at any stage of the lifecycle, including recovery.

4.4 Authentication & Authorization (Phase 9)

User registration always assigns the default USER role at the backend — clients cannot self-assign ADMIN. Passwords are hashed using BCryptPasswordEncoder before storage. On successful login, a JWT is issued containing the user's email (subject) and role as claims. A JwtAuthenticationFilter validates the token on each request and populates the Spring Security context accordingly. The API is stateless (no server-side session) and CSRF protection is disabled, consistent with token-based authentication.

The following role-based access control behavior was explicitly tested and verified on a protected admin endpoint:



4.5 Exception Handling (Phase 8)

All exceptions are handled centrally via a @RestControllerAdvice-based GlobalExceptionHandler, returning a consistent ErrorResponse structure (timestamp, status, error, message, path) instead of raw stack traces. Custom exceptions (e.g., KeyNotFoundException) map to appropriate HTTP status codes (e.g., 404 Not Found), and validation failures on request bodies are automatically converted to 400 Bad Request responses.

4.6 Audit Logging (Phase 10)

An AuditLoggingFilter runs after JWT authentication, so the authenticated user's identity is available for each recorded action. Every significant operation (e.g., key rotation, revocation, archival) is persisted in an audit_logs table capturing who performed the action, what action was taken, which endpoint was involved, when it occurred, and its result — providing traceability required for healthcare compliance and incident investigation.

4.7 Database Transactions & Consistency (Phase 14)

Multi-step key lifecycle operations (such as rotation, which updates one key version and creates another) are wrapped in @Transactional boundaries to guarantee atomicity — either all steps succeed, or none are applied, preventing inconsistent states such as a keyId with zero or multiple ACTIVE versions.

Test Step	Request / Action	Result
1. Create key	POST /api/keys — keyId: PHASE14-TX-001	id=21, version=1, status=ACTIVE
2. Rotate key	PATCH /api/keys/PHASE14-TX-001/rotate	id=22, version=2, status=ACTIVE
3. Verify DB state	Query all versions for PHASE14-TX-001	v1 → ROTATED, v2 → ACTIVE
4. Verify invariant	Count ACTIVE versions for keyId	Active count = 1 (confirmed)

Conclusion: This test confirms the core lifecycle invariant — only one ACTIVE version exists per keyId after rotation — holds under actual execution, validating the transactional design.

5. Experimental Results Summary

Test Area	Test Performed	Outcome
Encryption	AES-256-GCM encrypt/decrypt round-trip with random IV	Passed — plaintext correctly recovered; ciphertext differs per encryption
Authorization	Admin endpoint access with No JWT / USER JWT / ADMIN JWT	Passed — 401 / 403 / 200 responses exactly as expected
Key Rotation	Rotate active key and verify database state	Passed — old version ROTATED, new version ACTIVE, only 1 active count
Key Expiration	Scheduler-based expiry check on expiresAt	Passed — expired keys automatically transitioned to EXPIRED
Key Archival	Attempt to archive with an ACTIVE version present	Passed — system correctly blocks archival of active version
Emergency Recovery	Invalid state transitions (e.g., execute before approve)	Passed — invalid transitions rejected with conflict response
Exception Handling	Request for a non-existent key	Passed — returns structured 404 response, not a stack trace

6. Security Observations vs. Review 1 Limitations

Limitation (Review 1)	Current Status After Phases 1–14
Single point of failure in centralized key management	Key protection is now decoupled via a dedicated master-key-based protection layer, separate from raw storage.
Unauthorized modifications to keys	RBAC enforced and verified — non-admin JWTs are rejected (403) on privileged key operations.
Poor auditability of key operations	Every key lifecycle action is now captured by the Audit Logging module with actor, action, endpoint, timestamp, and result.
Manual/error-prone key lifecycle handling	Fully automated state machine for rotation, expiration (scheduler-driven), archival, and controlled recovery.

Blockchain-backed tamper-resistant logging (to fully eliminate the 'single point of failure' concern at the ledger level) remains planned for the next phase of development, as outlined below.

7. Remaining Work Plan

Development beyond this Mid-Term review follows the full project roadmap:

Stage	Phases	Focus
Before Review 3 (Result Validation)	15 – 18	Validation, key usage/retrieval hardening, advanced rotation, API documentation (Swagger/OpenAPI), testing, backend hardening — completes the backend.
Review 4 / 5 / 6 (Sem-7)	19 – 23	Blockchain fundamentals, blockchain architecture, smart contract development, backend ↔ blockchain integration, and blockchain verification.
Major Project II (Sem-8)	24 – 33	Full React.js frontend (authentication UI, dashboard, key management, audit UI, blockchain UI), user management, UI/UX polish, and full end-to-end integration.

8. Expected Outcome by Review 3

A validated, fully-documented, and thoroughly tested backend key-management engine — with Swagger-documented APIs and a hardened codebase — ready to receive blockchain integration in subsequent reviews, and ultimately a complete frontend in Major Project II.